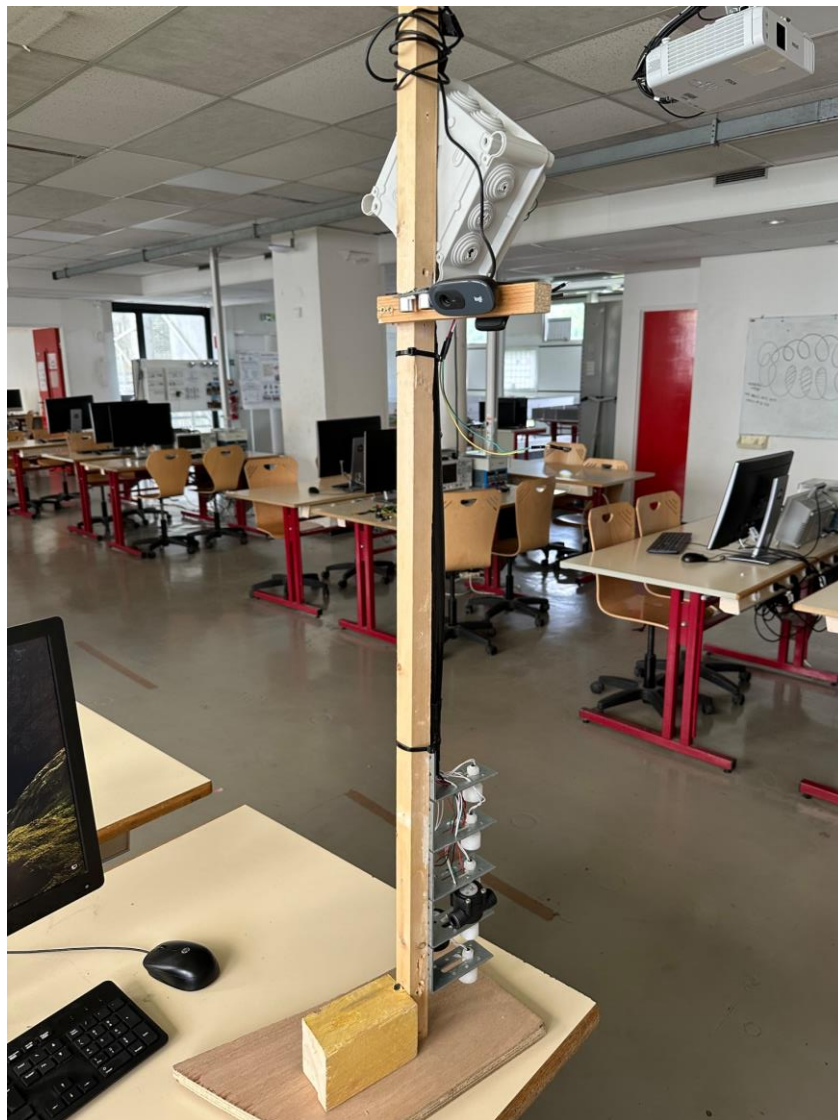


COLLIGNON Adrien
HENARD Félix
MONTEIRO Pacôme
MUGABIRE Elysee

FLOOD SENTINEL

Station hydrologique de
prévention des inondations



SOMMAIRE :

Table des matières

Contexte.....	3
Objectif	6
Etat de l'art	8
Composants	9
Résultats	15
Gestion	21
Conclusion et compétences	22
Remerciements	23
Bibliographie	24
Annexes	30
Guide d'utilisation	50

CONTEXTE :

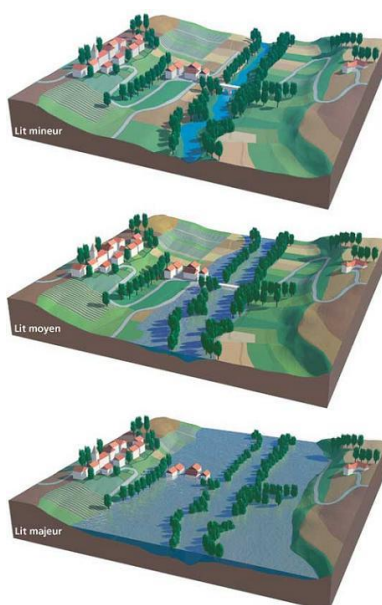


Les phénomènes météorologiques ne cessent de s'intensifier exponentiellement, et sont de plus en plus présents ainsi que violents de nos jours et cela ne risque pas s'améliorer avec le changement climatique. Les orages violents, les pluies diluviennes, la sécheresse de la terre et l'imperméabilisation des sols en ville augmentent notamment le risque d'inondations ou de crues.

En France la première source de catastrophes naturelles sont les inondations, 18,5 millions de français sont exposés à des risques d'inondations, ce qui représente aussi 54 % des actifs qui ont un emploi qui peut être exposé. Ainsi, depuis 1982 l'état français à indemnisé à hauteur de 24,8 milliards d'euros la population pour cause de crues et d'inondations [1].

Mais alors, qu'est-ce qu'une inondation ? Il en existe en réalité plusieurs causes :

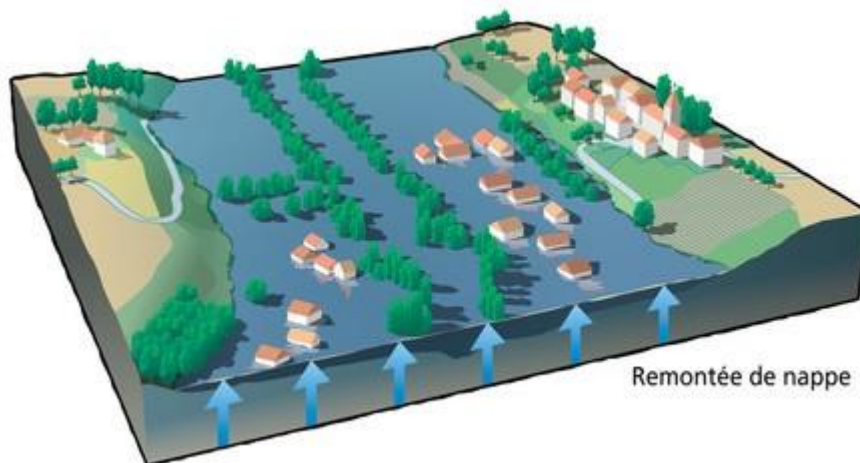
- le débordement d'un cours d'eau : lorsqu'un cours d'eau sort de son lit et inonde les alentours, c'est le cas le plus fréquent.



- le ruissellement en milieu urbain : l'imperméabilisation du sol empêche l'eau de s'infiltrer dans le sol et crée des torrents d'eau, de boue et d'objets charrier par les eaux et qui emprunte les rues jusqu'à rejoindre un réseau d'évacuation ou une rivière.



- la remontée de nappe phréatique : lors de pluie de longue durée, le niveau des nappes phréatiques peut monter jusqu'au débordement.



- la submersion : sur le littoral, le vent ou les tempêtes peuvent entraîner des vagues qui viennent inonder les zones côtières.
-



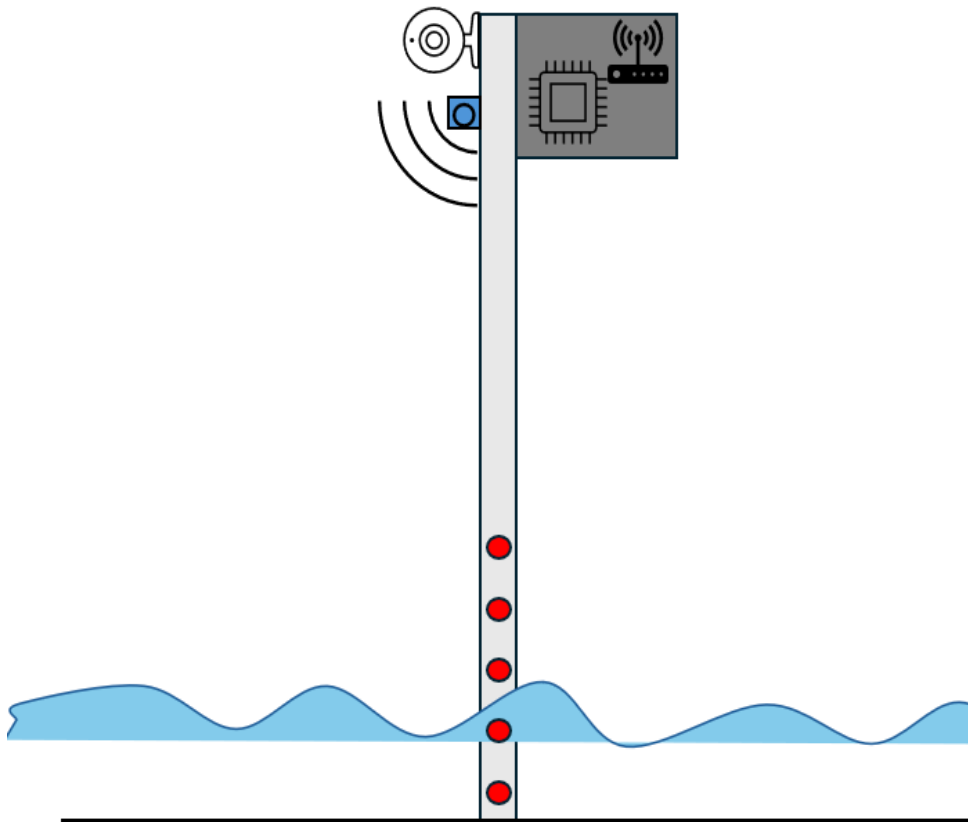
Ainsi la prévention et l'étude de ce risque météorologique qui peut toucher tout le monde, sur toute la planète, sont devenues très importantes pour les années à venir afin de limiter les dégâts et de protéger la population mondiale face à un phénomène qui peut être violent.

L'OBJECTIF DU PROJET :

Notre projet se tourne vers la réalisation d'une centrale miniature de captation d'informations, autour d'une rivière, d'une zone potentiellement inondable avec un cours d'eau, ou même en milieu urbain.

Nous souhaitons pouvoir visualiser les données recueillies par nos capteurs sur un site internet ou une application pour permettre de mieux informer et protéger la population en cas de montée des eaux. Notre projet a pour but de mieux comprendre et anticiper les inondations en créant une sorte de réseau de station afin de cartographier au mieux et limiter les dégâts. En effet, ce module se doit de pouvoir observer son environnement grâce à des capteurs. Ces derniers sont les suivants : une caméra, un capteur à ultrasons ou à ultrason, un débitmètre et enfin des capteurs de niveau à flotteur. Ces informations permettent ainsi d'analyser le déroulement d'une inondation et prévenir les habitants d'une potentielle crue. Ces données sont aussi intéressantes puisqu'elles peuvent permettre de comprendre le comportement des rivières avant et pendant les inondations. Nous souhaitons également que le système soit autonome énergétiquement et nous pensons ainsi installer un panneau solaire.

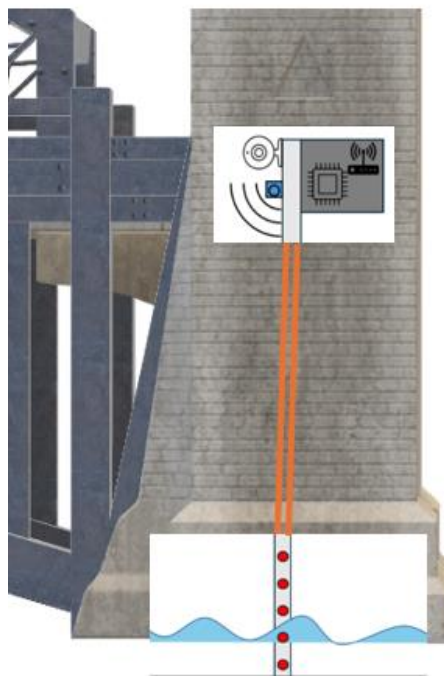
Nous avons statué sur un système qui est décrit par le schéma suivant :



On observe sur le schéma que l'on souhaite produire une tige qui puisse être installée en ville,

sur un mur ou avec un poteau. Ainsi au point le plus bas du tube on installe les capteurs de niveau qui permet de savoir s'il y a une présence d'eau mais également connaître le niveau d'eau. Avec les capteurs flotteurs, nous voudrions placer un débitmètre afin de pouvoir observer la vitesse de l'eau au niveau du lit de la rivière. Par la suite, on souhaite installer une caméra au point le plus haut afin que celle-ci calcule la vitesse de l'eau mais également observer la rivière au cours de sa vie. Avec les données de la caméra on peut alors imaginer par exemple un système de deep learning. Ainsi on pourrait prévoir les crues ou tout autre sorte de catastrophes et de risques concernant l'eau, à l'avance. Tout cela serait alors dirigé par un GPU autonome et intelligent (une Raspberry Pi 4, par exemple ; cette carte étant parfaite pour ce type d'utilisation).

L'adaptation de notre système pour son installation sur des ouvrages d'art, comme des ponts, dans le but de faire des mesures directement dans des cours d'eau est également envisageable. Il suffira simplement de garder la partie haute de notre système et de la fixer sur le pont avec le panneau solaire, pour ensuite fixer la partie basse avec la tige de capteurs sur les fondations de l'ouvrage, le tout relié par une connexion filaire ou bien sans fil. De même,



nous voulons aussi pouvoir visualiser les données récoltées par les différents capteurs pour proposer un suivi, jusqu'à même venir créer des alertes de prévention en cas de montée des eaux.

LES COMPOSANTS :

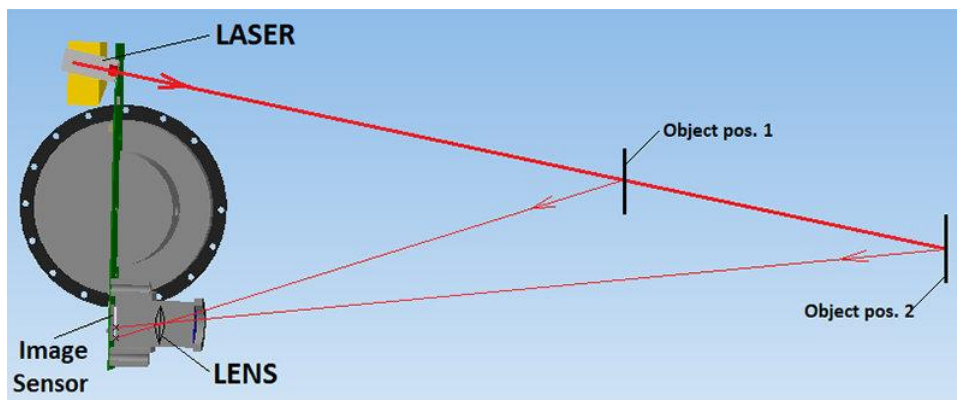


Raspberry pi 4 :

La carte Raspberry pi est une nano-ordinateur monocarte très polyvalente et très répandu, elle est utilisée principalement pour apprendre la programmation et pour utiliser des capteurs plus ou moins développés. La carte dispose de différents ports pour connecter différents composants et même des cartes annexes pour des utilisations particulières. Le premier avantage de la raspberry est sa taille, pas plus grande qu'une carte bancaire, ce qui la rend pratique pour des systèmes embarqués et compacts. De plus, la carte est à un prix abordable ce qui est pratique pour un système qui se veut accessible au plus grand nombre. La Raspberry est aussi suffisamment puissante pour alimenter tous les capteurs et ensuite traiter les données récoltées. La carte est équipée du bluetooth et d'une carte Wifi ce qui permet de pouvoir envoyer nos données sur un site internet ou autre. [4]

Notre choix s'est donc rapidement porté sur la Raspberry pour notre projet car elle regroupe toutes nos attentes : faible coût, basse consommation, polyvalence, petite taille et un grand nombre d'utilisateurs pour faciliter les recherches en cas de problèmes.

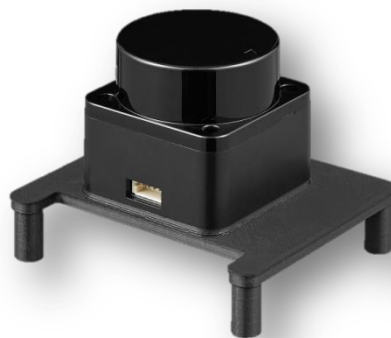
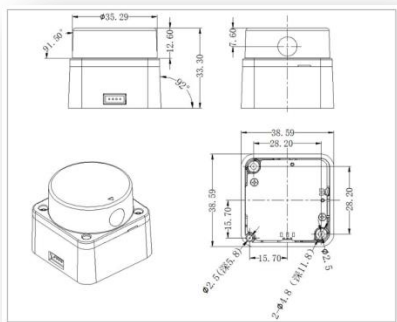
LIDAR (Light Detection And Ranging) :



Le LIDAR (*Laser Imaging Detection And Ranging*) est un capteur de mesure par temps de vol qui permet de connaître la position et la distance d'un objet. Le Lidar LD06 est très polyvalent et peut notamment s'utiliser très facilement avec un Raspberry PI. Le lidar est équipé d'un laser qui émet un rayon lumineux qui sera ensuite réfléchi sur un objet, et d'un capteur (récepteur) pour récupérer le faisceau réfléchi. Du fait de la vitesse de la lumière qui est constante, on peut donc connaître le temps qu'a mis le laser pour parcourir la distance aller-retour et ainsi connaître la distance réelle de l'objet [5]. Puisque le capteur est rotatif, nous n'allons réellement étudier qu'un angle de 120° en direction du Sud pour le capteur, permettant de cartographier ainsi la surface de l'eau. D'après les calculs, cela revient à mesurer une distance de 30 mètres, permettant ainsi de déterminer l'évolution de la surface, mais également les anomalies (comme les débris, coulées de boue, etc.). En outre, de par sa fréquence de mesures de 4500 Hz, il est possible de prendre 4500 mesures (points) différents par seconde. Mais la fréquence du laser utilisé étant de l'ordre des micro-ondes, elle ne peut donc pas interférer avec le capteur à ultrasons, ce qui permet une bonne complémentarité des deux. Le tout est utilisé via un script Python (voir en annexe).

Les caractéristiques de ce dernier sont les suivantes :

- Portée : 12 M
- Scan De 360°
- Fréquence Des Mesures : 4500 Hz
- Fréquence Des Scans : 10 Hz
- Scans Par Micro-Ondes Envoyées 10^{13} Hz environ

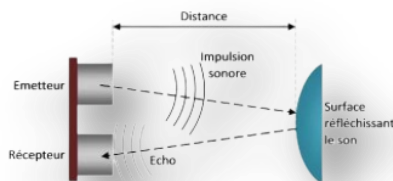


Capteur à ultrasons :

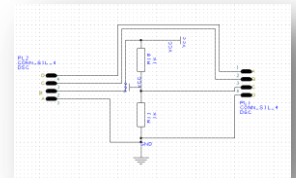


Nous avons choisi de prendre un capteur à ultrasons pour permettre de calculer la distance avec le niveau de l'eau. Il s'agit du capteur HC-SR04 qui se branche sur une Raspberry et qui permet de mesurer des distances de 2 à 400 cm ce qui sera largement suffisant pour notre prototype [6]. Dans des cas réels concernant les ponts, il faudrait utiliser un capteur US plus puissant (ou bien autre type de capteur), afin d'atteindre une mesure depuis une plus longue

distance ; ceci dit : pour notre prototype et pour les petits cours d'eau, ce capteur spécifique est tout à fait envisageable. Concrètement, le capteur envoie des signaux ultrasonores de l'ordre de 40 kHz que l'on va programmer toutes les 10 secondes et ensuite stocker ces données dans un fichier pour exploitation. De plus, pour permettre un meilleur fonctionnement, on utilise un amplificateur fait-maison (plaque PCB et résistances) afin d'augmenter le signal, mais aussi de réduire le bruit pouvant brouiller la mesure.



$$d_{obstacle}(cm) = \frac{T_{ultrason}(\mu s) \times v_{son}(m/s) \times 10^{-4}}{2}$$



La distance parcourue par les ultrasons est proportionnelle au temps du parcours. (Voir équation, en annexe, dans le code). Sur la Raspberry, nous allons procéder comme suit : pour déclencher une mesure, il faut présenter une impulsion "high" (3,3 V) d'au moins 10 μs sur l'entrée "Trig input". Le module émet alors une série de 8 impulsions ultra-soniques à 40 kHz, puis il attend le signal réfléchi. Lorsque celui-ci est détecté, il envoie un signal "high" sur la sortie "Output" (ou Echo), dont la durée est proportionnelle à la distance mesurée.

Caractéristiques techniques :

Capteur ultrason HC-SR04 :

Tension: 3.3 - 5V

Fréquence ultrasons : 40 kHz

Étendue mesure : 2-400 cm

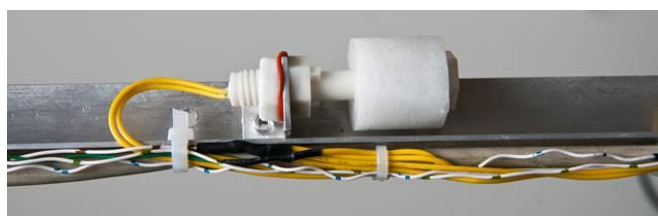
Angle de mesure : 15°

Une broche d'entrée : Trig (input) **GPIO25**

Une broche de sortie : Echo (output) **GPIO23**

Capteur de niveau :

Les capteurs de niveau d'eau permettent de relever la hauteur de l'eau notamment grâce à un flotteur mobile qui bouge sur une tige et qui ferme le circuit pour indiquer s'il y a de l'eau ou non [7]. L'avantage de ces capteurs avec flotteur c'est qu'ils peuvent être installés en circuit ouvrant ou en circuit fermant en fonction du sens dans lequel on place le capteur. Nous allons donc placer plusieurs capteurs sur une tige à différents niveaux et donc ainsi connaître le niveau d'eau lorsque l'eau va atteindre les capteurs et fermer le circuit. De plus, nous allons relier tous nos capteurs à une carte annexe branchée sur la Raspberry et ensuite extraire le niveau d'eau en connaissant la position de chaque capteur.



Capteur de débit :

Le capteur de débit permet de calculer le débit volumique d'eau grâce à la vitesse de rotation du rotor qui se trouve à l'intérieur. Lorsque l'eau rentre par la valve, l'eau entraîne le rotor magnétique et la vitesse de rotation varie selon le débit. Ensuite le capteur à effet Hall produira un signal à largeur d'impulsion qui permet de déduire le débit. On connecte le tout à notre raspberry pour pouvoir suivre les variations de débit et ensuite afficher les données sur une application ou site internet.



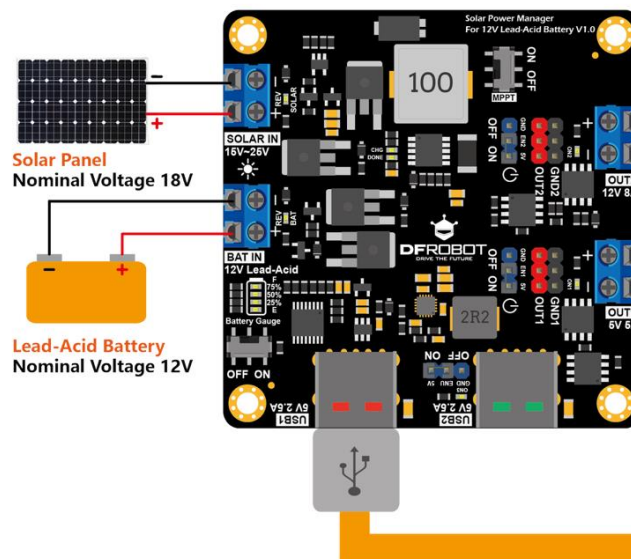
La WEBCAM :

On souhaite implémenter une webcam qui nous permettra de pouvoir calculer la vitesse d'écoulement de l'eau à la surface de la rivière. Ainsi cette webcam serait reliée à la raspberry. Pour éviter d'accroître nos dépenses, nous avons décidé d'utiliser une webcam HD. Ce choix vient également de la carte raspberry qui n'a pas la capacité de calcul nécessaire pour une qualité d'image supérieure. De plus, nous souhaitons stocker des images et des données récoltées et les fichiers HD sont également moins lourds et permettent une réduction des coûts ce qui est un critère pour notre prototype. Ainsi cette webcam sera commandée par un programme python qui gèrera le flux vidéo afin d'en extraire une photo et une vidéo qui sera par la suite analysée.



Alimentation énergétique :

Notre système se doit d'être autonome en énergie, aussi facile à installer et à manipuler. C'est pourquoi nous allons utiliser un panneau solaire et une batterie au plomb pour pouvoir assurer l'alimentation de la carte de jour comme de nuit et par tout temps. Pour pouvoir alimenter une raspberry avec un panneau solaire et une batterie, il faut utiliser une carte annexe qui s'occupe de gérer l'alimentation de la carte et la gestion de la batterie pour que le système fonctionne de manière optimale. Cette carte de gestion de l'énergie va pouvoir décider quand transmettre directement l'énergie produite par le panneau solaire et quand au contraire alimenter aussi un peu la batterie pour la recharger. La carte de gestion est uniquement compatible avec un panneau solaire de 100W et de 15-26 Vcc et quant à la batterie il faut utiliser une batterie au plomb de 12V, avec une capacité suffisante pour alimenter la raspberry ainsi que les capteurs la nuit et en cas de mauvais ensoleillement.[8]



Node Red :

Node Red est un logiciel de programmation avec des nœuds et que l'on peut relier facilement en un seul clic. Le logiciel permet de relier des périphériques matériels, des API ou des services en ligne facilement. Node Red est composé d'une palette proposant un large éventail de modules pré-programmés qu'il suffit d'importer mais on peut aussi programmer nous même notre fonction en JavaScript. Tout ceci se passe sur un navigateur et est parfaitement adapté pour l'utilisation de cartes arduino ou raspberry. Une fois notre flux créé, on peut déployer notre travail en un clic et node red propose un tableau de bord pour l'utilisateur afin d'afficher les données récoltées sur une interface très visuel et simple d'utilisation. De plus Node Red possède une communauté assez importante qui partage leurs projets (flux) directement sur le site internet, ce qui permet de prendre le logiciel



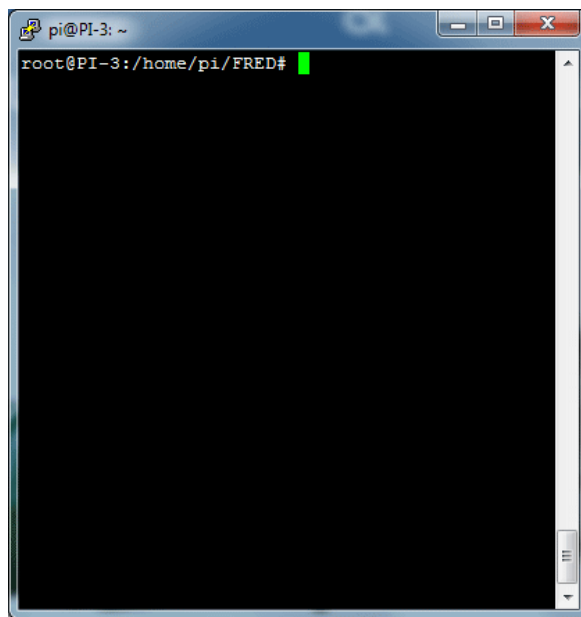
facilement en main en important les flux en JSON directement sur le navigateur [9]. Nous comptons utiliser Node Red pour créer une interface utilisateur et envoyer des messages de prévention. L'utilisateur pourra avoir accès à la position du flood sentinel, les débits, la hauteur de l'eau avec l'ultrason et les capteurs flotteurs mais aussi la vitesse de surface de l'eau avec la caméra.



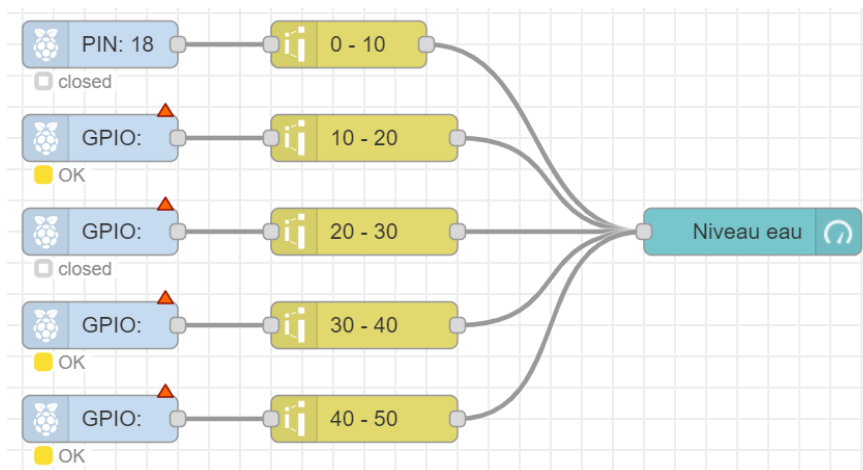
NOS RÉSULTATS :

Capteur de niveau :

Tout d'abord nous avons branché directement le capteur avec flotteur sur la carte raspberry mais lors de l'exécution du programme nous n'avons pas un résultat parfait. C'est-à-dire que le capteur retourne quasiment à chaque fois qu'il est à l'état haut même lorsqu'il est à l'état bas. Il y a donc une sorte de flou à cause d'un courant résiduel qui induit au capteur que le circuit est fermé alors qu'il est bien ouvert. C'est pourquoi nous rajoutons une résistance de pulldown pour obtenir un niveau logique (3,3V lorsque le circuit est fermé et 0V lorsque le circuit est ouvert) ainsi le capteur peut maintenant détecter la présence d'eau. Nous répétons cette opération pour plusieurs capteurs flotteurs pour pouvoir connaître le niveau d'eau étant donné que nous connaissons la position de chaque capteur sur le profilé. Il reste plus qu'à envoyer les données pour l'exploitation et la visualisation.

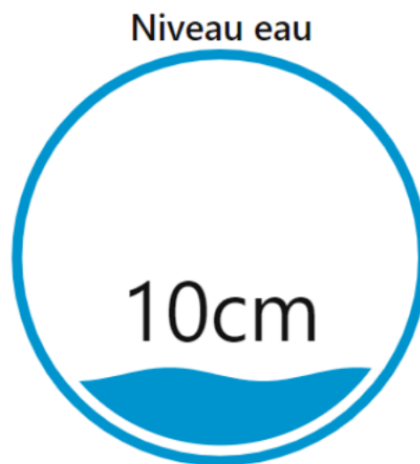


Une fois que nous sommes sur du bon fonctionnement des capteurs flotteurs, nous passons sur Node Red pour permettre à l'utilisateur de visualiser la hauteur :



Il suffit d'initialiser le pin de chacun des capteurs flotteurs et de paramétrer la hauteur à laquelle ils se trouvent dans les blocs jaune ci-dessus. Une fois que tout cela est fait, on ajoute une jauge pour permettre de visualiser les valeurs sur l'interface utilisateur.

Capteurs flotteur

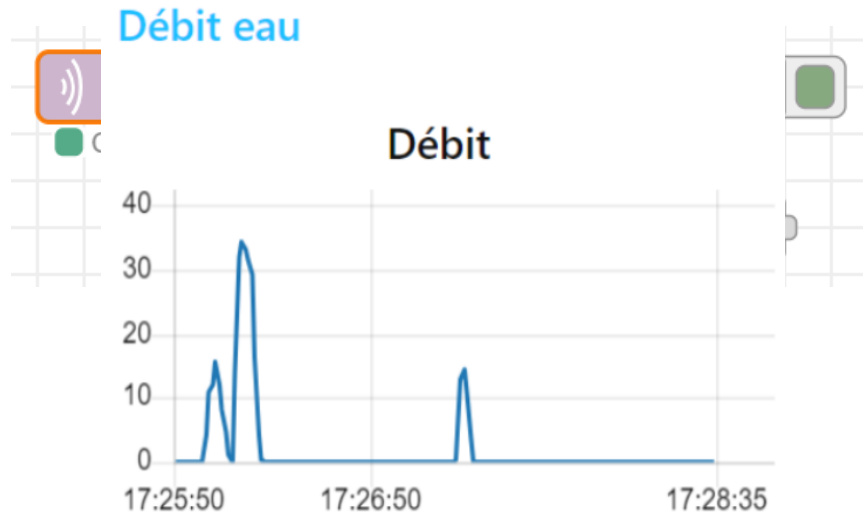


- Le débitmètre :

Comme pour les capteurs flotteurs, nous branchons dans un premier temps le débitmètre directement sur la carte raspberry puis nous trouvons un programme permettant de retourner la valeur en direct du débit en L/min. Une fois le programme fonctionnel, il faut trouver un moyen d'envoyer les valeurs obtenues vers notre tableau de bord Node Red. Ainsi pour se faire on utilise un protocole de messagerie basé sur un protocole TCP/IP qui se nomme MQTT et qui permet la communication du programme avec Node Red. On doit donc installer sur la

raspberry la librairie mosquitto pour pouvoir ensuite l'utiliser au cœur de notre programme en paramétrant le port et le canal d'envoi de nos valeurs. Node Red dispose de base un bloc permettant de recevoir des informations via MQTT donc nous allons l'utiliser :

Une fois la liaison faite entre le programme et Node Red, on ajoute un graphique pour permettre de visualiser les variations du débit sur l'interface utilisateur.



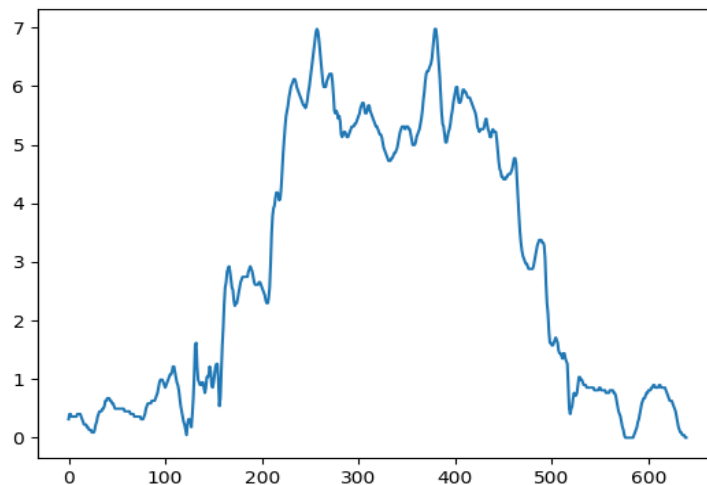
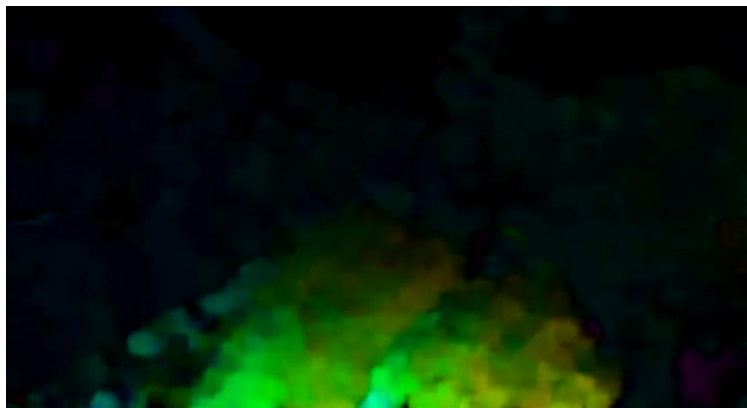
Sur notre graphique, on peut voir en abscisse le temps et en ordonnée le débit en L/min, nous allons ajouter des légendes plus tard.

- La webcam :

Le code python "programme.py" a pour but de calculer la vitesse à la surface de l'eau. Ce système est couplé à un débitmètre qui se trouve au fond du lit de la rivière afin de calculer la vitesse de l'eau au niveau le plus profond. Ainsi le code fonctionne grâce à un raspberry PI 4 et une webcam. Le code est composé de plusieurs fonctions principales. On y trouve une fonction qui permet de visionner le flux vidéo afin de l'enregistrer. Une vidéo est par la suite analysée image par image. On détermine ainsi les vecteurs d'accélération grâce aux fonctions contenues dans la bibliothèque OpenCV : https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html. Ainsi on utilise principalement la fonction `calcOpticalFlowFarneback` : https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html, qui permet avec une analyse image par image d'obtenir une matrice couleur de ce type, et par rapport à :



On voit ainsi que l'eau en mouvement est d'écrite par la couleur. Chaque couleur est composée de plusieurs vecteur, RGB. Ainsi chaque composante couleur correspond à une entité du vecteur accélération avec la magnitude et l'angle, ce qui est pour nous le plus important. Ainsi une fois ces informations extraites des vidéo on peut alors obtenir la vitesse sur chaque pixel de la vidéo. On obtient ainsi des moyennes de vitesse sur les lignes de pixel afin d'obtenir des vitesses en fonction de la position.



On peut ainsi observer ici que sur la ligne 300 la vitesse au long de la ligne est différente en chaque point.

Ainsi avec ces données on peut les enregistrer dans un fichier CSV. Avec ce fichier CSV on peut afficher les valeurs sur le site internet Node RED.

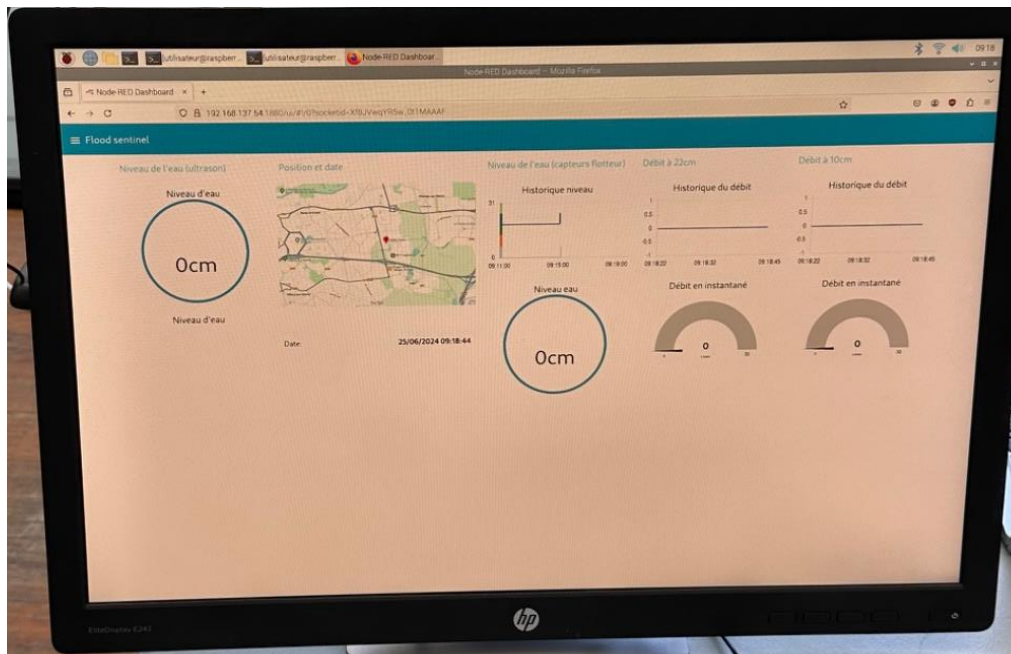
- Capteur ultrasons :

Voici le type de résultats obtenus, pour un affiche similaire aux capteurs flotteurs à posteriori sur le site :

```
+-----+
|  Mesure de distance par le capteur ultrasonore HC-SR04  |
+-----+
Entrez un nombre de repetitions de mesure : 15
La distance est de : 76.3 cm
La distance est de : 77.9 cm
La distance est de : 75.7 cm
La distance est de : 76.4 cm
La distance est de : 76.4 cm
La distance est de : 77.1 cm
La distance est de : 75.1 cm
La distance est de : 75.9 cm
La distance est de : 75.9 cm
La distance est de : 74.9 cm
La distance est de : 75.0 cm
La distance est de : 75.5 cm
```

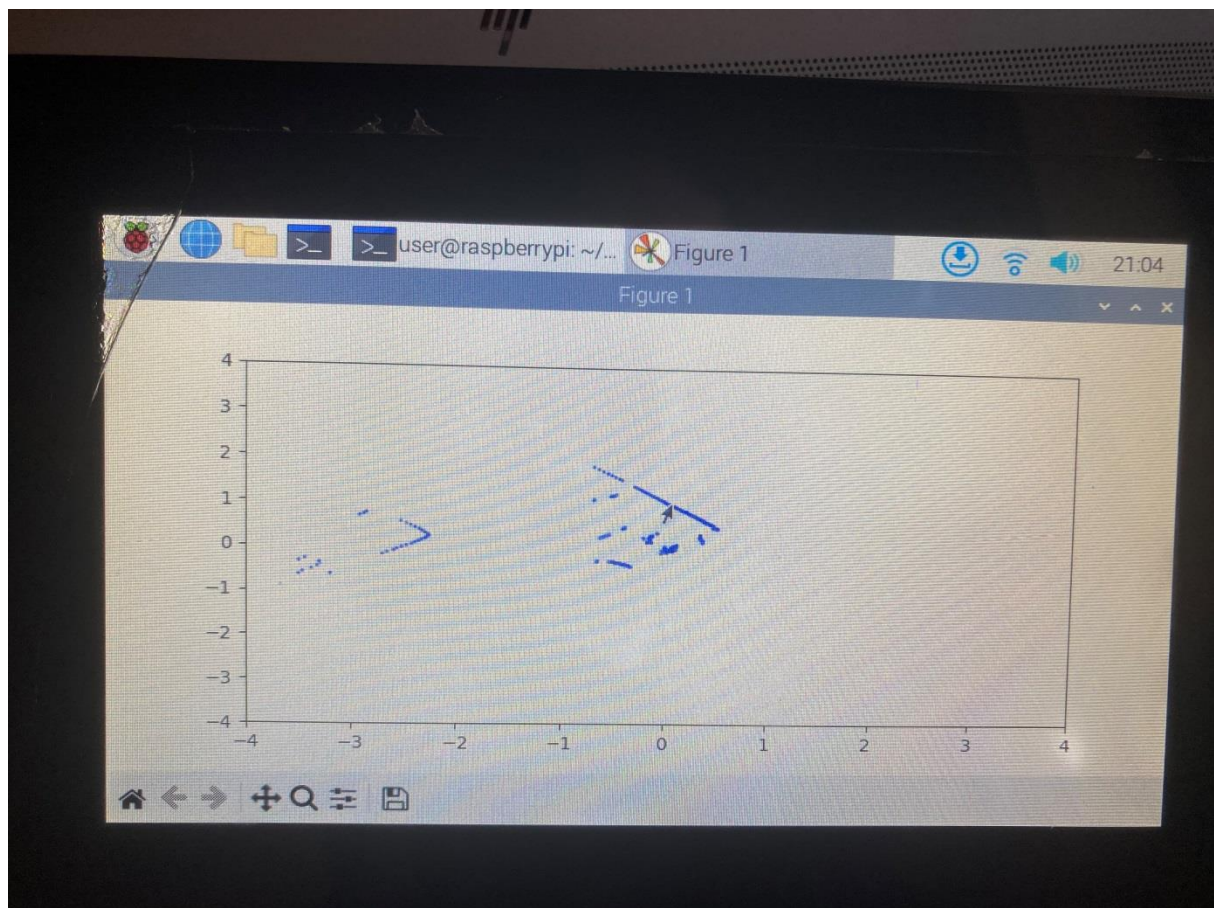
- Le site internet :

Comme évoqué précédemment, nous allons faire une interface utilisateur pour permettre de visualiser les informations des capteurs de manière simple et efficace grâce à Node RED. Pour cela il suffit de créer différents flux pour permettre de recevoir les informations de nos capteurs. Nous devons aussi paramétrer les graphiques et la position de chacun d'eux sur l'interface ainsi que de créer les alertes par notification. De plus, il a fallu utiliser la bibliothèque python MQTT pour permettre l'envoi des données des débitmètres et du capteur à ultrason, afin d'obtenir cette interface.



- Capteur ultrasons :

Voici le type de résultats obtenus, pour le lidar :



GESTION DE PROJET :

Pour mener à bien le projet nous avons tout d'abord commencé par découvrir le thème, faire des recherches pour nous imprégner et nous donner des idées sur ce que nous pouvons faire pour rendre le meilleur projet possible. Une fois cela effectué, nous avons cherché les composants à utiliser pour notre projet et nous les avons commandés. Il a fallu cadrer un peu le projet en se répartissant les rôles dès le début, une personne travaillant sur le traitement de l'image, une personne sur LIDAR/ultrason, une autre sur les débitmètres et enfin une personne sur les capteurs flotteurs. Lorsqu'une personne avait fini sa partie, il allait aider sur une autre tâche ou en commencer une nouvelle comme faire le prototype ou concevoir l'interface utilisateur. La répartition des tâches devait être plutôt équitable mais au vu des problèmes rencontrés sur le LIDAR, une partie du groupe a été mobilisée quasiment à temps plein sur cette tâche qui a finalement, à la fin du projet, abouti, après avoir recherché de nombreuses pistes. Ainsi nous avons pu avancer correctement sur le projet malgré une équipe pas toujours présente mais nous sommes contents du rendu et de cette première expérience professionnelle sur une longue durée.

CONCLUSION :

Notre station Flood Sentinel fonctionne correctement et nous avons répondu aux principales exigences du projet comme de créer une station installable aussi bien en milieu urbain ou sur un pont pour étudier les cours d'eau. Tout cela à moindre coût et de manière autonome en prenant des mesures en instantanée pour le débit et en différé pour le niveau de l'eau et la vitesse de surface de l'eau. L'interface utilisateur permet une bonne visualisation et correspond à nos attentes.

COMPETENCES ESIEE :

Notre projet Flood Sentinel permet une accessibilité au plus grand nombre que ce soit à un particulier ou à des communes pour l'étude, l'analyse et la prévention des risques d'inondation. Notre station est facile à installer et permet d'aller plus loin que nos concurrents directs, Vortex-io, qui propose uniquement la surveillance de cours d'eau et cela à un prix bien plus élevé que le nôtre, en effet nous pouvons installer notre système dans n'importe quel endroit et à un prix limité. Nous avons utilisé des composants faciles d'accès comme une raspberry et des capteurs faciles à implémenter, les plus grandes difficultés sont dans l'utilisation du LIDAR, l'algorithme du traitement de l'image et la conception de l'interface utilisateur. Grâce aux compétences acquises lors de cette première année à l'ESIEE nous avons mobilisé les compétences en termes d'informatique avec python pour les différents capteurs, mais aussi JAVA pour l'interface utilisateur et enfin des compétences managériales pour mener à bien le projet.

REMERCIEMENTS :

Nous souhaitons également remercier Mme Dokladalova de nous avoir proposé ce projet mais également de nous avoir accompagné et aidé durant toute la durée de ce projet. Nous voulions aussi remercier M Bues et M Pinto de nous avoir aidé et permis de répondre à nos demandes de composants. Nous voulions également remercier le BDE ATLAS et l'ESIEE Space pour nous avoir permis d'emprunter du matériel afin de réaliser le prototype. Enfin nous souhaiterions remercier les personnes qui nous ont aidé lors du déroulement de la période de projet, M Rimbault, Mme Cevaer et Mme Durand.

BIBLIOGRAPHIE :

- [1] : <https://www.georisques.gouv.fr/minformer-sur-un-risque/inondation>
- [2] : <https://www.carnot-tsn.fr/floria-detection-inondations/>
- [3] : <https://www.vigicrues.gouv.fr/>
- [4] : <https://www.raspberrypi-france.fr/guide/quest-ce-quun-raspberry-pi/>
- [5] : <https://www.generationrobots.com/blog/fr/qu-est-ce-que-la-technologie-lidar/>
- [6] : <https://raspberry-lab.fr/Composants/Mesure-de-distance-avec-HC-SR04-Raspberry-Francais/>
- [7] : <https://docs.rs-online.com/8e45/A700000010378611.pdf>
- [8] : https://wiki.dfrobot.com/Solar_Power_Manager_For_12V_Lead-Acid_Battery_SKU_DFR0580
- [9] : <https://nodered.org/>

Autres ressources

https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html
https://pytorch.org/hub/intelisl_midas_v2/
<https://matplotlib.org/3.5.3/index.html>
<https://numpy.org/>
<https://opencv.org/>

ANNEXES :

Code pour le capteur à ultrasons :

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

#Configuration des ports :
Trig = 23 # Entrée Trig du HC-SR04 branchée au GPIO 23
Echo = 24 # Sortie Echo du HC-SR04 branchée au GPIO 24

#Et de leur direction :
GPIO.setup(Trig,GPIO.OUT)
GPIO.setup(Echo,GPIO.IN)

v = 340 #Vitesse du son dans l'air = 340 m/s

def distance() :

    #Initialisation :
    GPIO.output(Trig, False)
    repet = input("Entrez un nombre de repetitions de mesure : ")
    N = input("Entrez un intervalle de temps pour la prise de mesure : ")
    repet = int(repet)
    N = int(N)

    #Programme :
    for x in range(repet): # On prend la mesure "repet" fois

        time.sleep(N) # On la prend toute les N seconde(s)

        GPIO.output(Trig, True)
        time.sleep(0.00001)
        GPIO.output(Trig, False)

        while GPIO.input(Echo)==0: ## Emission de l'US
            debutImpulsion = time.time()

        while GPIO.input(Echo)==1: ## Retour de l'echo de l'US
            finImpulsion = time.time()

        distance = round((finImpulsion - debutImpulsion) * v * 100 / 2, 1)

        print("La distance mesurée est de : ", distance, " cm.")

    GPIO.cleanup()
    #return()
```

Lidar ;

```
#!/usr/bin/env python3

import numpy as np
import matplotlib.pyplot as plt
import serial
from enum import Enum
import struct

# -----
# System Constants
# -----
# Serial port of the LIDAR unit
SERIAL_PORT = "/dev/ttyAMA0"
# At the default rotation speed (~3600 deg/s) the system outputs about
# 480 measurements in a full rotation. We want to plot at least this
# many in order to get a 360 degree plot
MEASUREMENTS_PER_PLOT = 480
# System will plot between +/-PLOT_MAX_RANGE on both X and Y axis
PLOT_MAX_RANGE = 4.0 # in meters
# Set the plot area to autoscale to the max distance in X or Y
PLOT_AUTO_RANGE = False
# Show the confidence as a colour gradient on the plot
PLOT_CONFIDENCE = True
# Set the colour gradient to use. See this URL for options:
# https://matplotlib.org/stable/gallery/color/colormap_reference.html
PLOT_CONFIDENCE_COLOUR_MAP = "bwr_r"
# Enable debug messages
PRINT_DEBUG = False
# -----
# Main Packet Format
# -----
# All fields are little endian
# Header (1 byte) = 0x54
# Length (1 byte) = 0x2C (assumed to be constant)
# Speed (2 bytes) Rotation speed in degrees per second
# Start angle (2 bytes) divide by 100.0 to get angle in degrees
# Data Measurements (MEASUREMENT_LENGTH * 3 bytes)
# See "Format of each data measurement" below
# Stop angle (2 bytes) divide by 100.0 to get angle in degrees
# Timestamp (2 bytes) In milliseconds
# CRC (1 bytes) Poly: 0x4D, Initial Value: 0x00, Final Xor Value: 0x00
# Input reflected: False, Result Reflected: False
# http://www.sunshine2k.de/coding/javascript/crc/crc_js.html
# Format of each data measurement
# Distance (2 bytes) # In millimeters
# Confidence (1 byte)
# -----
# Packet format constants
# -----
# These do not vary
PACKET_LENGTH = 47
MEASUREMENT_LENGTH = 12
MESSAGE_FORMAT = "<xBHH" + "HB" * MEASUREMENT_LENGTH + "HHB"

State = Enum("State", ["SYNC0", "SYNC1", "SYNC2", "LOCKED", "UPDATE_PLOT"])

def parse_lidar_data(data):
```

```

# Extract data
length, speed, start_angle, *pos_data, stop_angle, timestamp, crc = \
    struct.unpack(MESSAGE_FORMAT, data)
# Scale values
start_angle = float(start_angle) / 100.0
stop_angle = float(stop_angle) / 100.0
# Unwrap angle if needed and calculate angle step size
if stop_angle < start_angle:
    stop_angle += 360.0
step_size = (stop_angle - start_angle) / (MEASUREMENT_LENGTH - 1)
# Get the angle for each measurement in packet
angle = [start_angle + step_size * i for i in
range(0, MEASUREMENT_LENGTH)]
distance = pos_data[0::2] # in millimeters
confidence = pos_data[1::2]
if PRINT_DEBUG:
    print(length, speed, start_angle, *pos_data, stop_angle, timestamp,
crc)
return list(zip(angle, distance, confidence))

def get_xyc_data(measurements):
    # Unpack the tuples
    angle = np.array([measurement[0] for measurement in measurements])
    distance = np.array([measurement[1] for measurement in measurements])
    confidence = np.array([measurement[2] for measurement in measurements])
    # Convert to cartesian coordinates in meters
    x = np.sin(np.radians(angle)) * (distance / 1000.0)
    y = np.cos(np.radians(angle)) * (distance / 1000.0)
    return x, y, confidence

running = True

def on_plot_close(event):
    global running
    running = False

if __name__ == "__main__":
    # Connect up to the LIDAR serial port
    lidar_serial = serial.Serial(SERIAL_PORT, 230400, timeout=0.5)

    # Set up initial state
    measurements = []
    data = b''
    state = State.SYNC0

    # Set up matplotlib plot
    plt.ion()
    # Force a square aspect ratio
    plt.rcParams['figure.figsize'] = [10, 10]
    plt.rcParams['lines.markersize'] = 2.0
    if PLOT_CONFIDENCE:
        graph = plt.scatter([], [], c=[], marker=".", vmin=0,
                             vmax=255, cmap=PLOT_CONFIDENCE_COLOUR_MAP)
    else:
        graph = plt.plot([], [], ".")[0]
    # Set up the program to shutdown when the plot is closed
    graph.figure.canvas.mpl_connect('close_event', on_plot_close)
    # Limit to +/- PLOT_MAX_RANGE meters
    plt.xlim(-PLOT_MAX_RANGE, PLOT_MAX_RANGE)
    plt.ylim(-PLOT_MAX_RANGE, PLOT_MAX_RANGE)

```

```

# Main state machine
while running:
    # Find 1st header byte
    if state == State.SYNC0:
        data = b''
        measurements = []
        if lidar_serial.read() == b'\x54':
            data = b'\x54'
            state = State.SYNC1
    # Find 2nd header byte
    # Technically this is the length field but the packet length
    # is fixed, so it can be treated as a constant.
    elif state == State.SYNC1:
        if lidar_serial.read() == b'\x2C':
            state = State.SYNC2
            data += b'\x2C'
        else:
            state = State.SYNC0
    # Read remainder of the packet (PACKET_LENGTH minus the 2 header
    # bytes we have already read).
    elif state == State.SYNC2:
        data += lidar_serial.read(PACKET_LENGTH - 2)
        if len(data) != PACKET_LENGTH:
            state = State.SYNC0
            continue
        measurements += parse_lidar_data(data)
        state = State.LOCKED
    elif state == State.LOCKED:
        data = lidar_serial.read(PACKET_LENGTH)
        if data[0] != 0x54 or len(data) != PACKET_LENGTH:
            print("WARNING: Serial sync lost")
            state = State.SYNC0
            continue
        measurements += parse_lidar_data(data)
        if len(measurements) > MEASUREMENTS_PER_PLOT:
            state = State.UPDATE_PLOT
    elif state == State.UPDATE_PLOT:
        x, y, c = get_xyc_data(measurements)
        # Work out max coordinate, and set the scale based on this.
        # Force a 1:1 aspect ratio
        if PLOT_AUTO_RANGE:
            mav_val = max([max(abs(x)), max(abs(y))]) * 1.2
            plt.xlim(-mav_val, mav_val)
            plt.ylim(-mav_val, mav_val)
        # Clear the previous data
        graph.remove()
        # Plot the new data
        if PLOT_CONFIDENCE:
            graph = plt.scatter(x, y, c=c, marker=".",
                               vmin=0, vmax=255,
                               cmap=PLOT_CONFIDENCE_COLOUR_MAP)
        else:
            graph = plt.plot(x, y, 'b.') [0]
        # Show the new data
        plt.pause(0.00001)
        # Get the next packet
        state = State.LOCKED
        measurements = []

```

Autre exemple de code :

```
#!/usr/bin/env python3
import numpy as np
import matplotlib.pyplot as plt
import serial
from enum import Enum
import struct

# Configuration selon les caractéristiques techniques :
SERIAL_PORT = "/dev/ttyAMA0"
MEASUREMENTS_PER_PLOT = 480
PLOT_MAX_RANGE = 4.0 # en mètres
PLOT_AUTO_RANGE = False
PLOT_CONFIDENCE = True
PLOT_CONFIDENCE_COLOUR_MAP = "bwr_r"
PRINT_DEBUG = False
PACKET_LENGTH = 47
MEASUREMENT_LENGTH = 12
MESSAGE_FORMAT = "<xBHH" + "HB" * MEASUREMENT_LENGTH + "HHB"

State = Enum("State", ["SYNC0", "SYNC1", "SYNC2", "LOCKED", "UPDATE_PLOT"])

def parse_lidar_data(data):
    # Extraction data
    length, speed, start_angle, *pos_data, stop_angle, timestamp, crc = \
        struct.unpack(MESSAGE_FORMAT, data)
    # Valeurs
    start_angle = float(start_angle) / 100.0
    stop_angle = float(stop_angle) / 100.0
    # Calculs d'angles
    if stop_angle < start_angle:
        stop_angle += 360.0
    step_size = (stop_angle - start_angle) / (MEASUREMENT_LENGTH - 1)
    # Angle pour chaque paquets
    angle = [start_angle + step_size * i for i in range(0, MEASUREMENT_LENGTH)]
    distance = pos_data[0::2] # en mm
    confidence = pos_data[1::2]
    if PRINT_DEBUG:
        print(length, speed, start_angle, *pos_data, stop_angle, timestamp, crc)
    return list(zip(angle, distance, confidence))

def get_xyc_data(measurements):
    # Tuples
    angle = np.array([measurement[0] for measurement in measurements])
    distance = np.array([measurement[1] for measurement in measurements])
    confidence = np.array([measurement[2] for measurement in measurements])
    # Conversion aux coordonnées cartésiennes en mètres
    x = np.sin(np.radians(angle)) * (distance / 1000.0)
    y = np.cos(np.radians(angle)) * (distance / 1000.0)
    return x, y, confidence

running = True
```

Programme capteurs flotteurs pour test :

```
import RPi.GPIO as GPIO
import time

DEBUG = 0;

#-----[ CABLAGE ]-----

# Cable de la sonde de 0(3%) a 5(100%)
Sonde = [5, 7, 8, 9, 10];

#-----[ Initialisation ]-----

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

for i in range(0,5):
    GPIO.setup(Sonde[i], GPIO.IN)

duration=100

#-----[ Main ]-----

start_time=time.time()
while( (time.time() - start_time) < duration ):
    time.sleep(1)
    print( "\n\n\n")

    for i in range(0,5):

        if (i == 0):
            P = "100"
        elif (i == 1):
            P = " 75"
        elif (i == 2):
            P = " 50"
        elif (i == 3):
            P = " 25"
```

```

elif (i == 4):
    P = " 3"

if GPIO.input(Sonde[i]) == True:
    if DEBUG :
        print( " ",P,"% |      | GPIO ",Sonde[i])
    else:
        print( " ",P,"% |      |")
else:
    if DEBUG :
        print( " ",P,"% |#####| GPIO ",Sonde[i])
    else:
        print( " ",P,"% |#####|")

print( "      -----")
print( "\n\n\n")
exit

```

Programme pour les débitmètre et envoi Node-RED :

```
from gpiozero import DigitalInputDevice
import time
import paho.mqtt.client as mqtt

import csv

FLOW_SENSOR_GPIO = 22 # Utilisez la broche GPIO 22

pulse_count = 0
flow_rate = 0.0
last_time = time.time()

def count_pulse():
    global pulse_count
    pulse_count += 1

flow_sensor = DigitalInputDevice(FLOW_SENSOR_GPIO, pull_up=True)
flow_sensor.when_activated = count_pulse

client = mqtt.Client()
client.connect("localhost", 1883, 60) # Assurez-vous que le broker MQTT est installé et en cours
d'exécution sur localhost

try:
    while True:
        current_time = time.time()
        elapsed_time = current_time - last_time
        if elapsed_time >= 1:
            flow_rate = (pulse_count / 7.5) / elapsed_time # Calcul du débit en L/min
            pulse_count = 0
            last_time = current_time

            # Publier la valeur du débit sur MQTT
            client.publish("sensor/debit", flow_rate)
            print("Débit : {:.3f} L/min".format(flow_rate))

            time.sleep(0.1)
except KeyboardInterrupt:
    print("Arrêt du script")
```

```
finally:  
    client.disconnect()
```

Programme pour le capteur ultrason et envoi Node-RED :

```
# Utilisation du programme :  
# source ~/myenv/bin/activate  
# sudo ~/myenv/bin/python distance.py  
  
import time  
import RPi.GPIO as GPIO  
import paho.mqtt.client as mqtt  
  
import csv  
  
# Configuration des broches GPIO  
TRIG = 25 # Entrée Trig du HC-SR04 branchée au GPIO 23  
ECHO = 23 # Sortie Echo du HC-SR04 branchée au GPIO 25  
  
# Initialisation du GPIO  
GPIO.setmode(GPIO.BCM)  
GPIO.setup(TRIG, GPIO.OUT)  
GPIO.setup(ECHO, GPIO.IN)  
  
v = 340 # Vitesse du son dans l'air = 340 m/s  
  
# Initialisation :  
# (Utilisation de "repet" si besoin d'un nombre précis de répétitions)  
# repet = input("Entrez un nombre de repetitions de mesure : ")  
N = input("Entrez un intervalle de temps pour la prise de mesure : ")  
# repet = int(repet)  
N = int(N)  
  
# Mesure de la distance  
def measure_distance():  
    time.sleep(N)  
    GPIO.output(TRIG, True)  
    time.sleep(0.00001)  
    GPIO.output(TRIG, False)  
  
    start_time = time.time()  
    stop_time = time.time()
```

```

while GPIO.input(ECHO) == 0: ## Emission de l'US
    start_time = time.time()

while GPIO.input(ECHO) == 1: ## Retour de l'echo de l'US
    stop_time = time.time()

time_elapsed = stop_time - start_time
distance = (time_elapsed * v * 100) / 2

return distance

# Envoi des données
# Broker MQTT installé et en cours d'exécution sur localhost
client = mqtt.Client()
client.connect("localhost", 1883, 60)

# Lancement du programme
try:
    while True:
        distance = measure_distance()
        client.publish("sensor/distance", distance)
        print(f"Distance : {distance:.2f} cm")
        time.sleep(1)
except KeyboardInterrupt:
    print("Mesure arrêtée")
    GPIO.cleanup()

```

Programme pour la avoir un retour vidéo du flou optique en directe :

```

import numpy as np
import cv2 as cv
from matplotlib.pyplot import *

def aff_vitesse_eau():

    cap = cv.VideoCapture(0)
    ret, frame1 = cap.read()
    prvs = cv.cvtColor(frame1, cv.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)

```

```

hsv[..., 1] = 255

nbr_frame=0

while(1):

    nbr_frame=nbr_frame+1

    ret, frame2 = cap.read()
    if not ret:
        print('No frames grabbed!')
        break

    next = cv.cvtColor(frame2, cv.COLOR_BGR2GRAY)
    flow = cv.calcOpticalFlowFarneback(prvs, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
    mag, ang = cv.cartToPolar(flow[..., 0], flow[..., 1])
    hsv[..., 0] = ang*180/np.pi/2
    hsv[..., 2] = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX)
    bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

    cv.imshow('Vitesse surface',bgr)

    k = cv.waitKey(30) & 0xff
    if k == 27:
        break
    elif k == ord('s'):
        cv.imwrite('opticalfb.png', frame2)
        cv.imwrite('opticalhsv.png', bgr)
    prvs = next

    cv.destroyAllWindows()

aff_vitesse_eau()

```

Programme pour la avoir un retour vidéo du flou optique avec la vidéo préenregistrés :

```

import numpy as np
import cv2 as cv
from matplotlib.pyplot import *

```

```

def aff_vitesse_eau(name):

    cap = cv.VideoCapture(cv.samples.findFile(name))
    ret, frame1 = cap.read()
    prvs = cv.cvtColor(frame1, cv.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255

    nbr_frame=0

    while(1):

        nbr_frame=nbr_frame+1

        ret, frame2 = cap.read()
        if not ret:
            print('No frames grabbed!')
            break

        next = cv.cvtColor(frame2, cv.COLOR_BGR2GRAY)
        flow = cv.calcOpticalFlowFarneback(prvs, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
        mag, ang = cv.cartToPolar(flow[..., 0], flow[..., 1])
        hsv[..., 0] = ang*180/np.pi/2
        hsv[..., 2] = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX)
        bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

        cv.imshow('Vitesse surface',bgr)

        k = cv.waitKey(30) & 0xff
        if k == 27:
            break
        elif k == ord('s'):
            cv.imwrite('opticalfb.png', frame2)
            cv.imwrite('opticalhsv.png', bgr)
            prvs = next

    cv.destroyAllWindows()

aff_vitesse_eau("seq_5.mp4")

```

Programme fonctionnel de la webcam avec une vidéo préenregistrés :

```

import numpy as np
import cv2 as cv
from matplotlib.pyplot import *
import csv

def enregistrement(record_duration, where_file):
    webcam = cv.VideoCapture(0)
    largeur_image = int(webcam.get(cv.CAP_PROP_FRAME_WIDTH))
    hauteur_image = int(webcam.get(cv.CAP_PROP_FRAME_HEIGHT))
    fourcc = cv.VideoWriter_fourcc(*'MJPG')
    fichier = cv.VideoWriter(where_file, fourcc, 20, (largeur_image, hauteur_image))
    start_time_record = time.time()
    while(True):
        # Capture image par image
        ret, img = webcam.read()
        if ret==True:
            fichier.write(img)
            # Preparation de l'affichage de l'image
            #cv.imshow('Ma Webcam',img)
            # affichage et saisie d'un code clavier
            if cv.waitKey(1) & 0xFF == ord('q'):
                break
            if (time.time() - start_time_record) > record_duration :
                break
        else:
            break
    # Ne pas oublier de fermer le flux et la fenetre
    webcam.release()
    fichier.release()
    cv.destroyAllWindows()

def vitesse_eau_region_ligne(name, origine, dimension):

    cap = cv.VideoCapture(cv.samples.findFile(name))
    ret, frame1 = cap.read()
    prvs = cv.cvtColor(frame1, cv.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255

```

```

#enregistrement des images à afficher sur le site internet avec une sauvegarde avec la date et
l'heure dans un répertoire dédié

cv.imwrite(f"/home/utilisateur/Desktop/flood_sentinel/SAVE/PICTURE/PICTURE_{time.asctime(time.
ocaltime())}.png" , frame1)
cv.imwrite("/var/www/html/PICTURE.png" , frame1)

#définition des variables
nbr_frame=0
(xo,yo)=origine
(xd,yd)=dimension
final=[]

while(1):

    nbr_frame=nbr_frame+1

    ret, frame2 = cap.read()
    if not ret:
        print('No frames grabbed!')
        break

    next = cv.cvtColor(frame2, cv.COLOR_BGR2GRAY)
    flow = cv.calcOpticalFlowFarneback(prvs, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
    mag, ang = cv.cartToPolar(flow[..., 0], flow[..., 1])
    hsv[..., 0] = ang*180/np.pi/2
    hsv[..., 2] = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX)
    bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

    #print(hsv.shape)

    values=hsv[xo:xo+xd, yo:yo+yd, 2]
    values_TR=values.mean(axis=0)
    final.append(values_TR*0.15*0.30)
    final_TR=np.array(final).mean(axis=1)

    #print(np.array(final).size)
    #print(np.array(final_TR).size)

    k = cv.waitKey(30) & 0xff
    if k == 27:

```

```

        break
    elif k == ord('s'):
        cv.imwrite('opticalfb.png', frame2)
        cv.imwrite('opticalhsv.png', bgr)
    prvs = next

    cv.destroyAllWindows()

    return(final, final_TR, nbr_frame)

def vitesse_eau_point_moyenne(name, origine, dimension):

    cap = cv.VideoCapture(cv.samples.findFile(name))
    ret, frame1 = cap.read()
    prvs = cv.cvtColor(frame1, cv.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255

    #définition des variables
    nbr_frame=0
    (xo,yo)=origine
    (xd,yd)=dimension
    final=[]

    while(1):

        nbr_frame=nbr_frame+1

        ret, frame2 = cap.read()
        if not ret:
            print('No frames grabbed!')
            break

        next = cv.cvtColor(frame2, cv.COLOR_BGR2GRAY)
        flow = cv.calcOpticalFlowFarneback(prvs, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
        mag, ang = cv.cartToPolar(flow[..., 0], flow[..., 1])
        hsv[..., 0] = ang*180/np.pi/2
        hsv[..., 2] = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX)
        bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

        #print(hsv.shape)

```

```

values=hsv[xo-round(xd/2):xo+round(xd/2), yo-round(yd/2):yo+round(yd/2), 2]
values_TR=values.mean(axis=0)
final.append(values_TR*0.15*0.30)
final_TR=np.array(final).mean(axis=1)

final_min=[]
final_max=[]
values_MM=hsv[..., 2]
values_MM=values_MM*0.15*0.30
#print(values_MM)
"""
for i in range(len(values_MM)):
    values_max=values_MM.max(axis=1).max(axis=0)
    values_min=values_MM.min(axis=1).min(axis=0)
    final_max.append(values_max)
    final_min.append(values_min)
final_min=np.array(final_min)
final_max=np.array(final_max)
"""

#print(np.array(final).size)
#print(np.array(final_TR).size)

k = cv.waitKey(30) & 0xff
if k == 27:
    break
elif k == ord('s'):
    cv.imwrite('opticalfb.png', frame2)
    cv.imwrite('opticalhsv.png', bgr)
prvs = next

cv.destroyAllWindows()

return(final, final_TR, final_max, final_min, nbr_frame)

def enregistrement_min_max(variable):
    clf()
    plot(variable[1])
    plot(variable[2])
    plot(variable[3])

```

```

savefig(f"/home/utilisateur/Desktop/flood_sentinel/SAVE/GRAPHIQUE/Graph_vitesse_pt{time.asctime(
time.localtime())}.png")
    savefig(f"/var/www/html/Graph2.png")

def affichage(variable):
    plot(variable)
    show()

def enregistrement_graph(variable):
    clf()
    plot(variable)

savefig(f"/home/utilisateur/Desktop/flood_sentinel/SAVE/GRAPHIQUE/Graph_vitesse_ligne{time.asctime(
time.localtime())}.png")
    savefig(f"/var/www/html/Graph.png")

def return_data(values):
    data = values
    with open('/home/utilisateur/Desktop/flood_sentinel/SAVE/DATA/VITESSE/data_VITESSE.csv',
'a') as f:
        writer = csv.writer(f, delimiter=';')
        writer.writerow(data)

def fonction(duration, duration_record, name_file, where_file, origine, dimension):
    enregistrement(duration_record, where_file)
    time.sleep(4)
    return_data(vitesse_eau_region_ligne(name_file, origine, dimension)[1])

def fonction2(name_file, origine, dimension):
    time.sleep(4)
    return_data(vitesse_eau_region_ligne(name_file, origine, dimension)[1])

#enregistrement(3, "/home/utilisateur/Desktop/flood sentinel/film2.avi")

#affichage(vitesse_eau_region_ligne("film2.avi", (300,0), (20,640))[1])

start_time=time.time()
duration_time=3601
while(True):

```

```

time.sleep(2)
#fonction(4, 2, "rec.avi", "/home/utilisateur/Desktop/flood_sentinel/PROGRAMME/rec.avi",
(300,0), (20,640))
fonction2("seq_5.mp4", (300,0), (20,640))
print("Enregistrement effectué.")
variable=vitesse_eau_region_ligne("seq_5.mp4", (300,0), (20,640))[1]
if (time.time() - start_time) > duration_time :
    break
enregistrement_graph(variable)
enregistrement_min_max(vitesse_eau_point_moyenne("seq_5.mp4", (370,240), (20,20)))
print("Valeurs enregistrer")

```

Programme final comprenant le code complet de détection et de sauvegarde des données recueilli par la webcam :

```

import numpy as np
import cv2 as cv
from matplotlib.pyplot import *
import csv

def enregistrement(record_duration, where_file):
    webcam = cv.VideoCapture(0)
    largeur_image = int(webcam.get(cv.CAP_PROP_FRAME_WIDTH))
    hauteur_image = int(webcam.get(cv.CAP_PROP_FRAME_HEIGHT))
    fourcc = cv.VideoWriter_fourcc(*'MJPG')
    fichier = cv.VideoWriter(where_file, fourcc,20,(largeur_image,hauteur_image))
    start_time_record=time.time()
    while(True):
        # Capture image par image
        ret, img = webcam.read()
        if ret==True:
            fichier.write(img)
            # Preparation de l'affichage de l'image
            #cv.imshow('Ma Webcam',img)
            # affichage et saisie d'un code clavier
            if cv.waitKey(1) & 0xFF == ord('q'):
                break
            if (time.time() - start_time_record) > record_duration :
                break
        else:

```

```

        break

# Ne pas oublier de fermer le flux et la fenetre
webcam.release()
fichier.release()
cv.destroyAllWindows()

def vitesse_eau_region_ligne(name, origine, dimension):

    cap = cv.VideoCapture(cv.samples.findFile(name))
    ret, frame1 = cap.read()
    prvs = cv.cvtColor(frame1, cv.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255

    #enregistrement des images à afficher sur le site internet avec une sauvegarde avec la date et
    l'heure dans un répertoire dédié

    cv.imwrite(f"/home/utilisateur/Desktop/flood_sentinel/SAVE/PICTURE/PICTURE_{time.asctime(time.
ocaltime())}.png" , frame1)
    cv.imwrite("/var/www/html/PICTURE.png" , frame1)

    #définition des variables
    nbr_frame=0
    (xo,yo)=origine
    (xd,yd)=dimension
    final=[]

    while(1):

        nbr_frame=nbr_frame+1

        ret, frame2 = cap.read()
        if not ret:
            print('No frames grabbed!')
            break

        next = cv.cvtColor(frame2, cv.COLOR_BGR2GRAY)
        flow = cv.calcOpticalFlowFarneback(prvs, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
        mag, ang = cv.cartToPolar(flow[..., 0], flow[..., 1])
        hsv[..., 0] = ang*180/np.pi/2
        hsv[..., 2] = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX)

```

```

bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

#print(hsv.shape)

values=hsv[xo:xo+xd, yo:yo+yd, 2]
values_TR=values.mean(axis=0)
final.append(values_TR*0.15*0.30)
final_TR=np.array(final).mean(axis=1)

#print(np.array(final).size)
#print(np.array(final_TR).size)

k = cv.waitKey(30) & 0xff
if k == 27:
    break
elif k == ord('s'):
    cv.imwrite('opticalfb.png', frame2)
    cv.imwrite('opticalhsv.png', bgr)
prvs = next

cv.destroyAllWindows()

return(final, final_TR, nbr_frame)

def vitesse_eau_point_moyenne(name, origine, dimension):

    cap = cv.VideoCapture(cv.samples.findFile(name))
    ret, frame1 = cap.read()
    prvs = cv.cvtColor(frame1, cv.COLOR_BGR2GRAY)
    hsv = np.zeros_like(frame1)
    hsv[..., 1] = 255

    #définition des variables
    nbr_frame=0
    (xo,yo)=origine
    (xd,yd)=dimension
    final=[]

    while(1):

        nbr_frame=nbr_frame+1

```

```

ret, frame2 = cap.read()
if not ret:
    print('No frames grabbed!')
    break

next = cv.cvtColor(frame2, cv.COLOR_BGR2GRAY)
flow = cv.calcOpticalFlowFarneback(prvs, next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
mag, ang = cv.cartToPolar(flow[..., 0], flow[..., 1])
hsv[..., 0] = ang*180/np.pi/2
hsv[..., 2] = cv.normalize(mag, None, 0, 255, cv.NORM_MINMAX)
bgr = cv.cvtColor(hsv, cv.COLOR_HSV2BGR)

#print(hsv.shape)

values=hsv[xo-round(xd/2):xo+round(xd/2), yo-round(yd/2):yo+round(yd/2), 2]
values_TR=values.mean(axis=0)
final.append(values_TR*0.15*0.30)
final_TR=np.array(final).mean(axis=0)

final_min=[]
final_max=[]
values_MM=hsv[..., 2]
values_MM=values_MM*0.15*0.30
#print(values_MM)
"""
for i in range(len(values_MM)):
    values_max=values_MM.max(axis=1).max(axis=0)
    values_min=values_MM.min(axis=1).min(axis=0)
    final_max.append(values_max)
    final_min.append(values_min)
final_min=np.array(final_min)
final_max=np.array(final_max)
"""

#print(np.array(final).size)
#print(np.array(final_TR).size)

k = cv.waitKey(30) & 0xff
if k == 27:
    break
elif k == ord('s'):

```

```

        cv.imwrite('opticalfb.png', frame2)
        cv.imwrite('opticalhsv.png', bgr)
        prvs = next

    cv.destroyAllWindows()

    return(final, final_TR, final_max, final_min, nbr_frame)

def enregistrement_min_max(variable):
    clf()
    plot(variable[1])
    plot(variable[2])
    plot(variable[3])

savefig(f"/home/utilisateur/Desktop/flood_sentinel/SAVE/GRAPHIQUE/Graph_vitesse_pt{time.asctime(time.localtime())}.png")
    savefig(f"/var/www/html/Graph2.png")

def affichage(variable):
    plot(variable)
    show()

def enregistrement_graph(variable):
    clf()
    plot(variable)

savefig(f"/home/utilisateur/Desktop/flood_sentinel/SAVE/GRAPHIQUE/Graph_vitesse_ligne{time.asctime(time.localtime())}.png")
    savefig(f"/var/www/html/Graph.png")

def return_data(values):
    data = values
    with open('/home/utilisateur/Desktop/flood_sentinel/SAVE/DATA/VITESSE/data_VITESSE.csv',
'a') as f:
        writer = csv.writer(f, delimiter=';')
        writer.writerow(data)

def fonction(duration, duration_record, name_file, where_file, origine, dimension):
    enregistrement(duration_record, where_file)
    time.sleep(4)
    return_data(vitesse_eau_region_ligne(name_file, origine, dimension)[1])

```

```

#enregistrement(3, "/home/utilisateur/Desktop/flood sentinel/film2.avi")

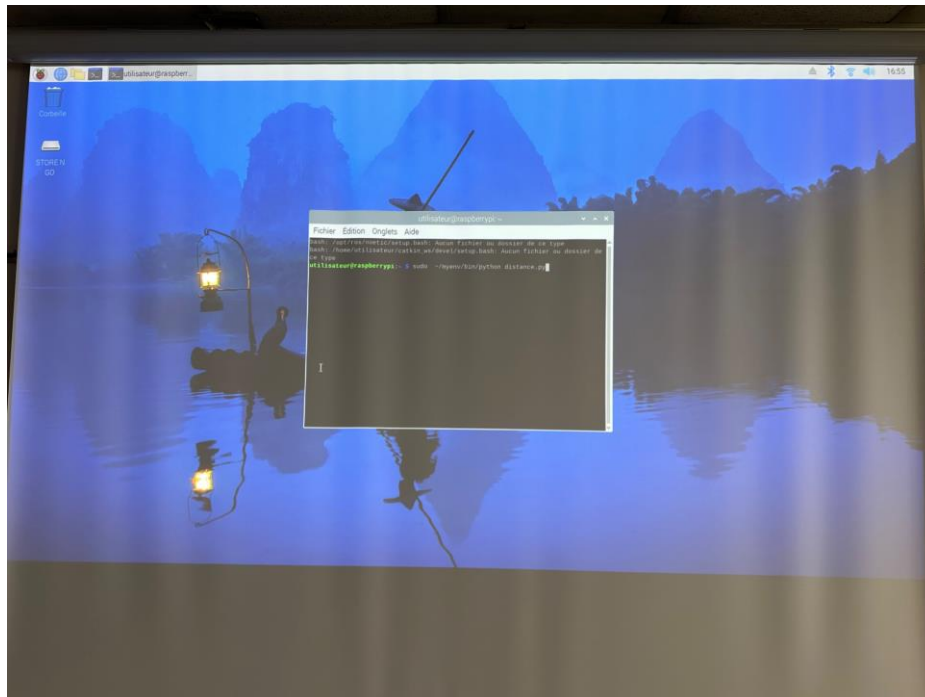
#affichage(vitesse_eau_region_ligne("film2.avi", (300,0), (20,640))[1])

start_time=time.time()
duration_time=3601
while(True):
    time.sleep(10)
    fonction(4, 2, "rec.avi", "/home/utilisateur/Desktop/flood sentinel/PROGRAMME/rec.avi", (300,0),
(20,640))
    print("Enregistrement effectué.")
    variable=vitesse_eau_region_ligne("rec.avi", (300,0), (20,640))[1]
    if (time.time() - start_time) > duration_time :
        break
    enregistrement_graph(variable)
    enregistrement_min_max(vitesse_eau_point_moyenne("rec.avi", (370,240), (20,20)))
    print("Valeurs enregistrer")

```

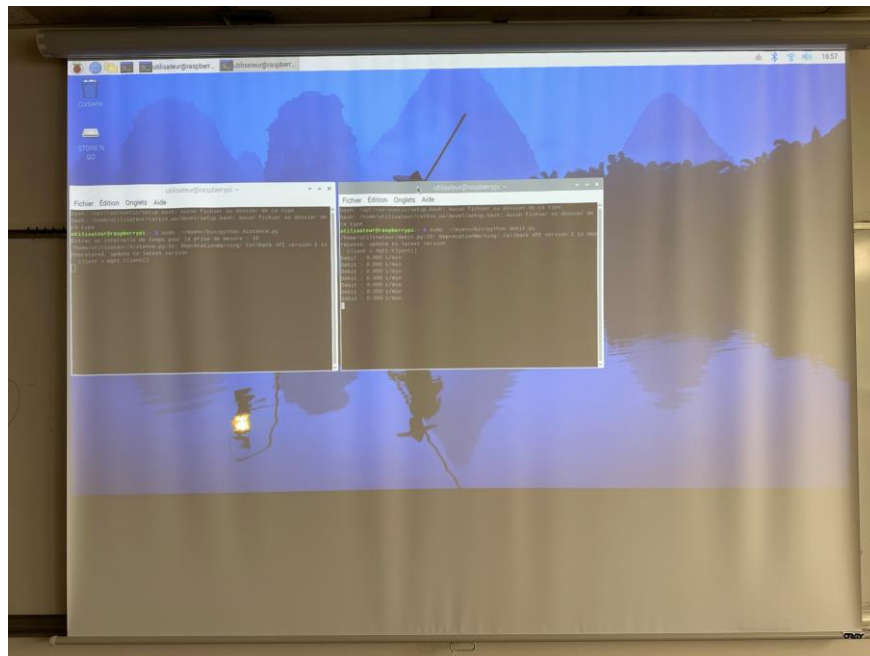

GUIDE UTILISATEUR :

1. Branché la raspberry et vérifier le bon branchement des capteurs.
2. La connecter à un réseau WIFI et récupérer son adresse IP.
3. Ouvrir le prompt et taper la commande :
sudo ~/myenv/bin/python distance.py



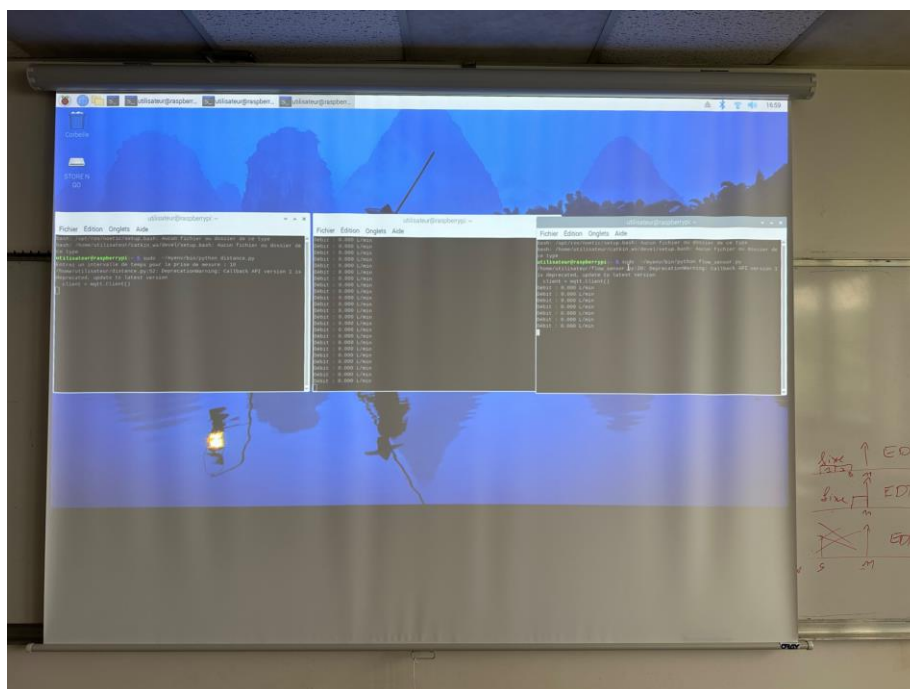
Ce qui permet de lancer le programme python qui utilise la capteur à ultrason et qui envoie les données directement a Node-RED.

4. Ensuite ouvrir un deuxième prompt en parallèle sans fermer l'autre sinon cela arrête le programme et donc l'envoi de données. Tapez cette fois si :
sudo ~/myenv/bin/python debit.py



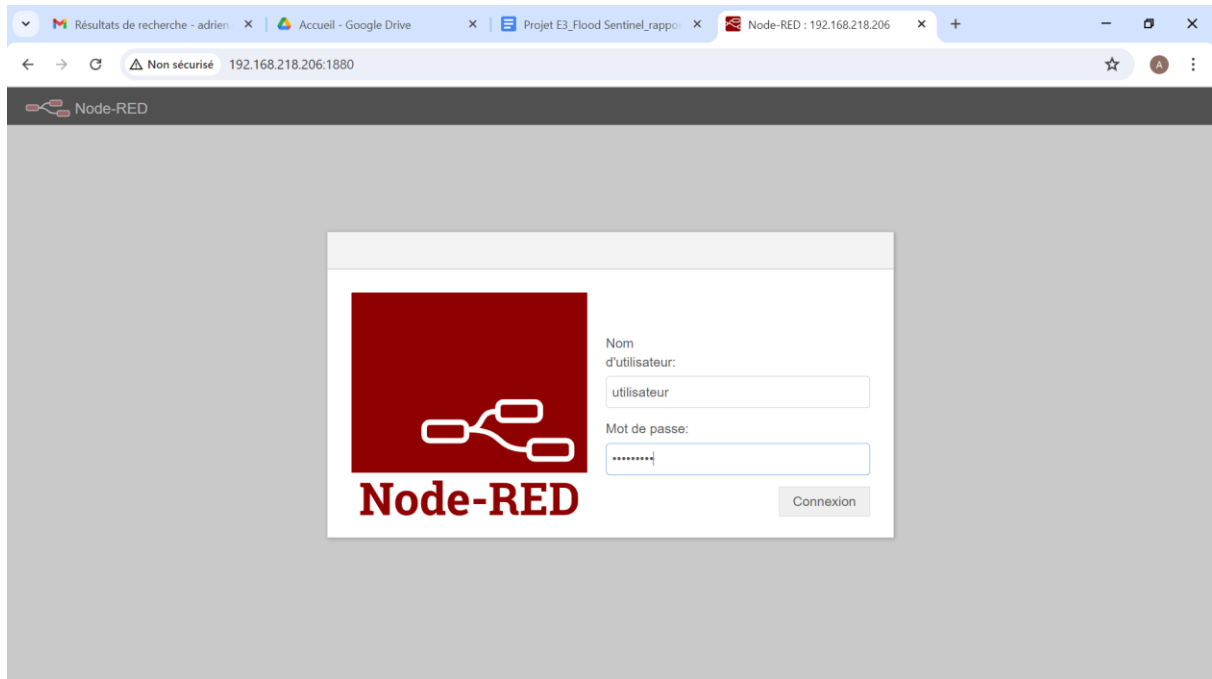
Ce qui affiche et envoi les valeurs du premier débitmètre.

5. Ouvrir un troisième prompt en parallèle des deux autres et taper :
`sudo ~/myenv/bin/python flow_sensor.py`



Ce qui affiche et envoi les valeurs du deuxième débitmètre.

6. Pour visualiser l'interface de programmation il suffit d'avoir un appareil connecté sur le même réseau et de rentrer dans la barre de recherche de navigateur :
- <ip raspberry>:1880.*
- Par exemple sur le réseau ESIEE l'IP de la raspberry est : 192.168.218.206.

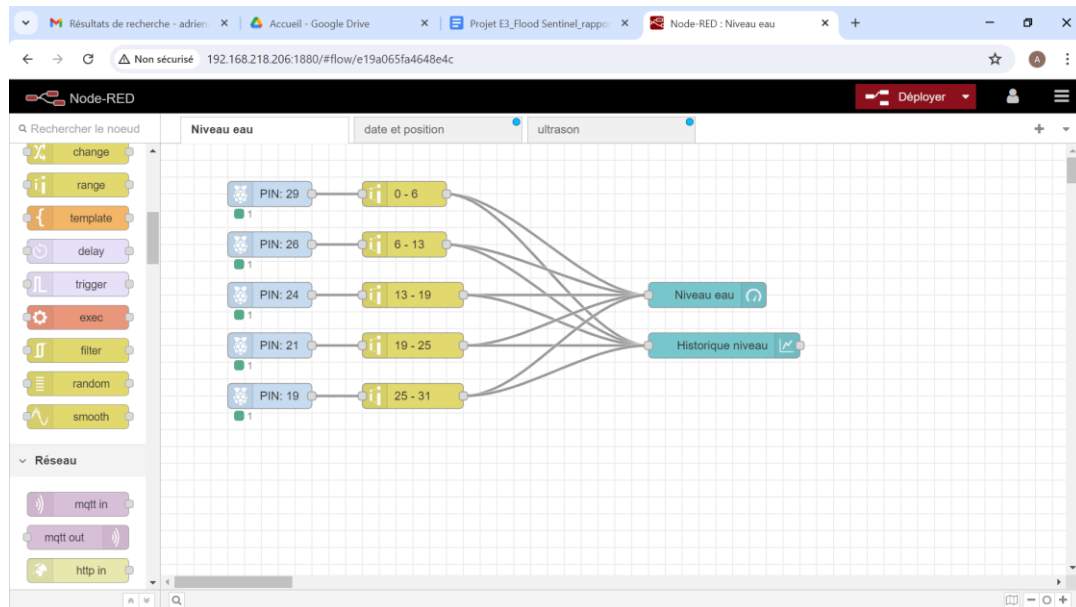


Pour rentrer sur l'interface de programmation il faut rentrer :

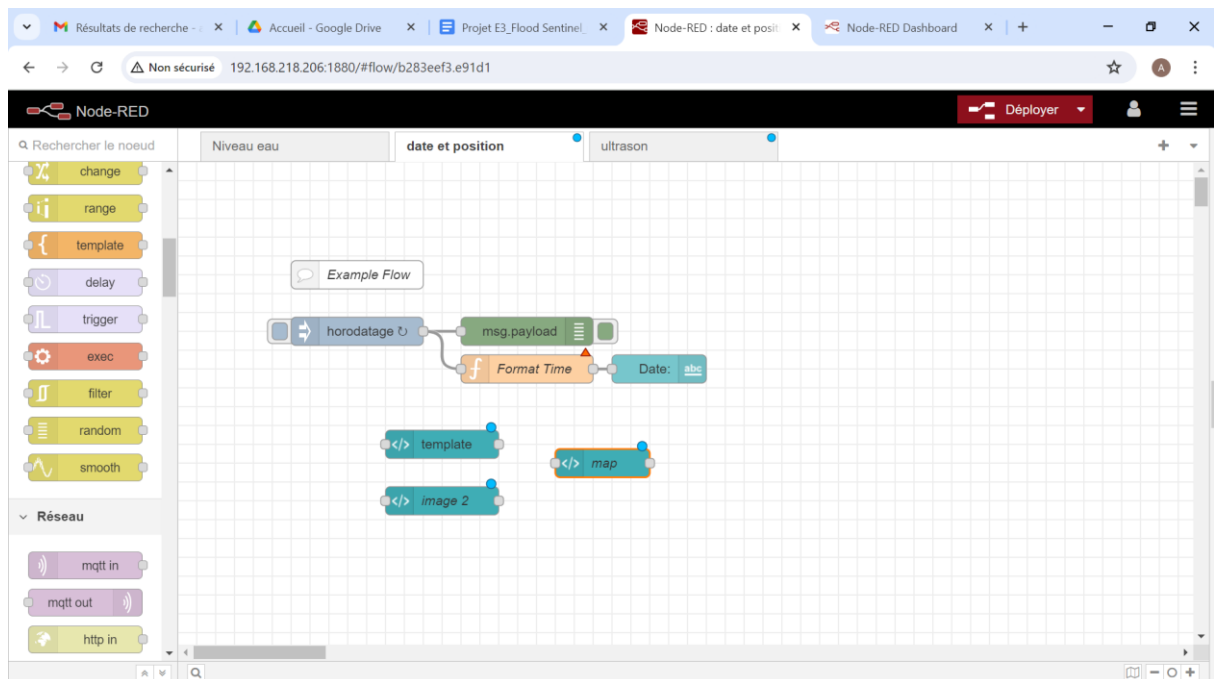
Utilisateur : utilisateur

Mot de passe : esiee2024

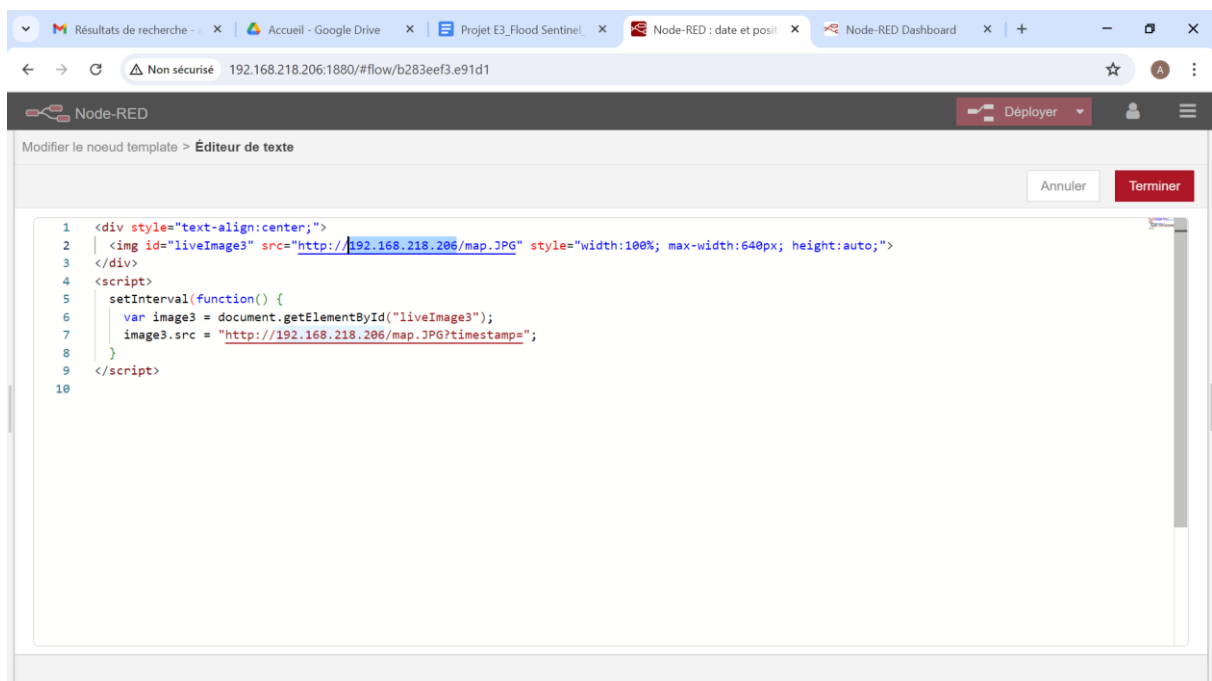
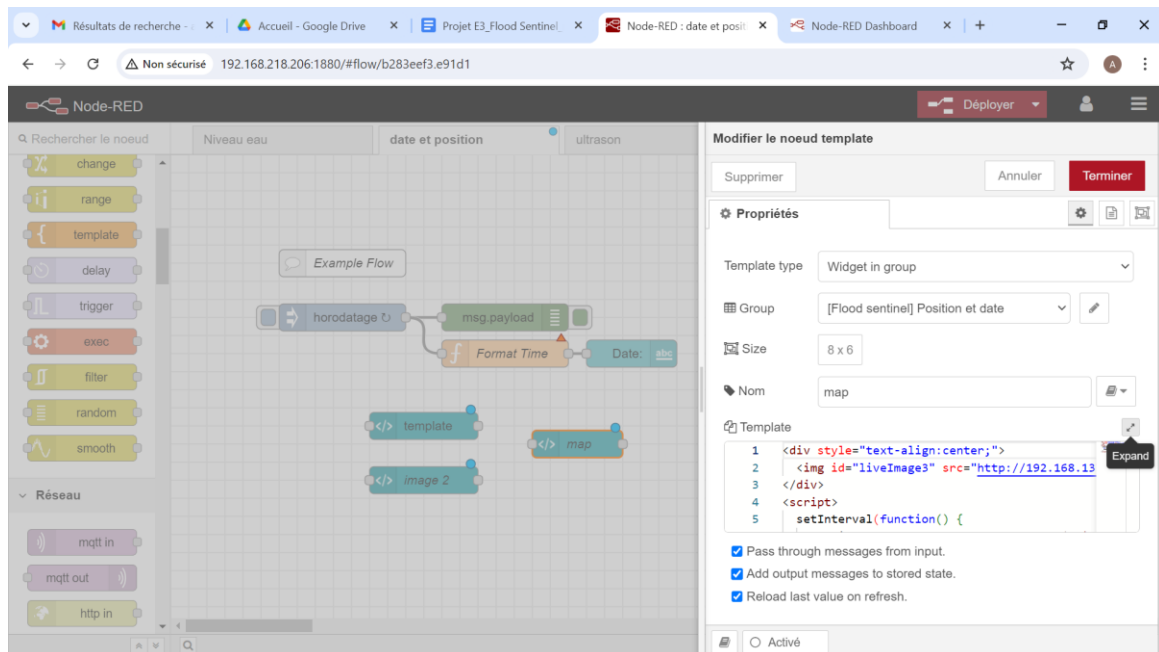
7. Ensuite vous arrivez dans l'interface de programmation :



8. Pour permettre d'afficher la carte il faut aller dans le flux date et position :

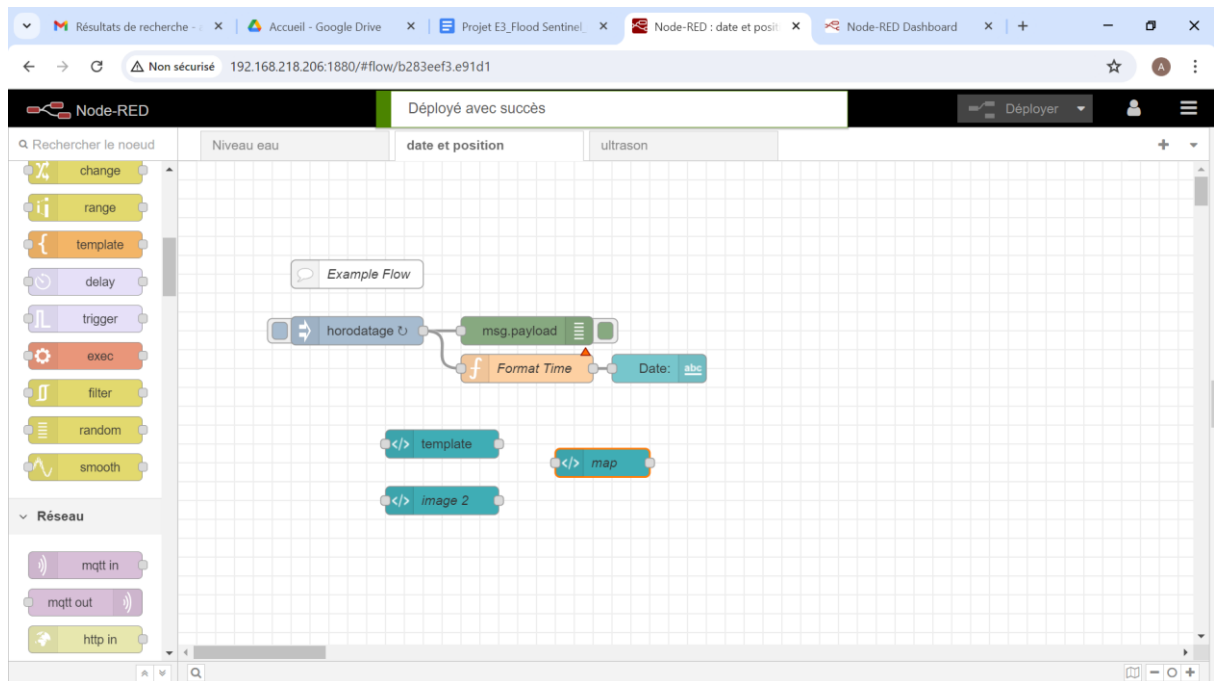


9. Il faut double cliquer sur map et modifier l'adresse IP de la carte si besoin dans le code en cliquant sur expand :

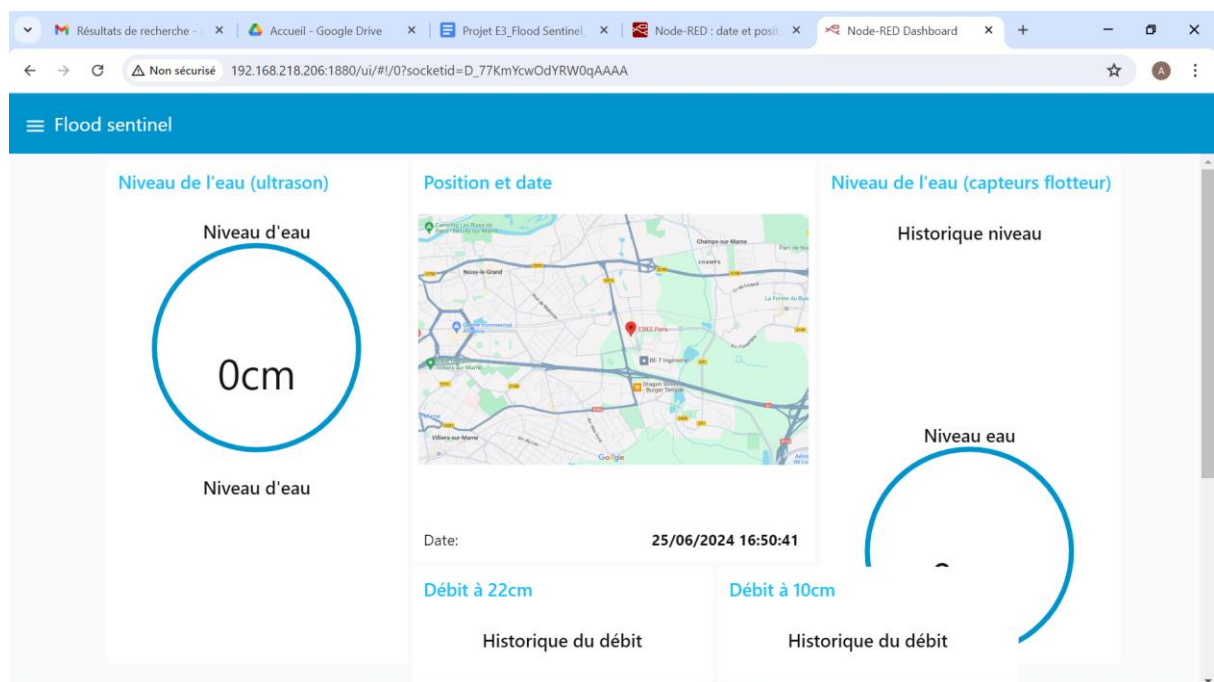


Ici l'adresse IP est correcte donc la carte s'affiche bien.

10. Une fois l'adresse IP modifiée il suffit de cliquer sur terminer pour sauvegarder et ensuite cliquez sur déployer le flux en haut à droite sur Node-RED. (lors du déploiement il y a une erreur mais cliquez sur confirmer).



11. Pour afficher cette fois ci l'interface utilisateur il faut taper dans la barre de recherche : `<ip raspberry>:1880/ui`.
Dans notre exemple : 192.168.218.206:1880/ui



L'interface apparaît et est accessible sur tous les appareils connectés à ce réseau WIFI.

Si vous avez fait toutes ces étapes vous devriez voir les variations du débit si vous soufflez dans les débitmètres et voir les valeurs du niveau de l'eau si vous montez les capteurs flotteurs.